

JOBSHEET 3

Mata Kuliah Pemrograman Berorientasi Objek Enkapsulasi pada Pemrograman Berorientasi Objek



Nama : Rachmah Nur Chotimah
NIM : 254107060052
Kelas : SIB 2F

3. Percobaan

3.1 Percobaan 1 – Tanpa Enkapsulasi

Didalam percobaan enkapsulasi, buatlah class Motor yang memiliki atribut platNomor, isMesinOn (bernilai true jika mesin sedang menyala dan false jika tidak menyala), dan kecepatan serta method displayStatus() untuk menampilkan status motor. UML class diagram class Motor adalah sebagai berikut:

Motor
+ platNomor: String
+ isMesinOn: Boolean
+ kecepatan: int
+displayStatus(): void

Hasil Praktikum

```
public class Motor{
    public String platNomor;
    public boolean isMesinon;
    public int kecepatan;

    public void displayStatus () {
        System.out.println("Plat Nomor: " + this.platNomor);

        if (isMesinOn) {
            System.out.println ("Mesin On");
        }else{
            System. out.println ("Mesin Off");
        }

        System.out.println ("Kecepatan:" + this.kecepatan);
        System. out.println ("=====");
    }
}

public class MotorDemo {
    public static void main(String[] args) {
        Motor motor1 = new Motor () ;
        motor1.displayStatus () ;

        motor1.platNomor = "B 0838 XZ";
        Motor motor1 = new Motor ();
        motor1.displayStatus ();
        motor1.platNomor = "B 0838 XZ";
        int kecepatanBaru = 50;
        if (!motor1.isMesinon && kecepatanBaru > 0) {
            System.out.println ("Kecepatan tidak boleh lebih dari 0 jika mesin off");
        }else{
            motor1.kecepatan = kecepatanBaru;
```

```

    }
    motor1.displayStatus ();

    Motor motor2 = new Motor () ;
    motor2.platNomor = "N 9840 AB";
    motor2.isMesinon = true;
    kecepatanBaru = 40;
    if (!motor2.isMesinon && kecepatanBaru > 0) {
        System.out.println ("Kecepatan tidak boleh lebih dari 0 jika
        mesin off");
    }else{
        motor2. kecepatan = kecepatanBaru;
    }

    Motor motor3 = new Motor () ;
    motor3.platNomor = "D 8343 CV";
    kecepatanBaru = 60;
    if(!motor3.isMesinon && kecepatanBaru > 0) {
        System.out.println ("Kecepatan tidak boleh lebih dari 0 jika mesin
        off");
    }else{
        motor3. kecepatan = kecepatanBaru;
    }
    motor3.displayStatus ();
}
}

```

3.2 Percobaan 2 – Enkapsulasi

1. Bayangkan bahwa developer baru ingat bahwa seharusnya kecepatan tidak boleh lebih dari 0 jika status mesin tidak menyala setelah membuat 20 objek motor di MotorDemo.java, 10 objek motor di MotorDemo2.java, 25 objek MotorDemo3.java? Pengecekan harus dilakukan 55 kali.
2. Lalu, bagaimana kita bisa memperbaiki class Motor diatas agar dapat digunakan dengan baik? Di sinilah pentingnya melakukan enkapsulasi dalam pemrograman berorientasi objek. Struktur internal class Motor harus disembunyikan dari class lain. Pada OOP, konsep enkapsulasi diimplementasikan dengan cara:
 - a. Menyembunyikan atribut internal (platNomor, isMesinOn, dan kecepatan) dari luar/class lain dengan mengubah access level modifier menjadi private
 - b. Menyediakan setter dan getter untuk memanipulasi dan mengakses nilai atribut tersebut

Motor
- platNomor: String - isMesinOn: Boolean

- kecepatan: int
+displayStatus(): void +setPlatNomor(platNomor:String): void +getPlatNomor(): String +setIsMesinOn(isMesinOn:boolean): void +getIsMesinOn(): boolean +setKecepatan(kecepatan:int): void +getKecepatan(): int

Hasil Praktikum

```

public class Motor {

    private String platNomor;
    private boolean isMesinOn;
    private int kecepatan;

    public void displayStatus() {
        System.out.println("Plat Nomor: " + this.platNomor);
        if (isMesinOn) {
            System.out.println("Mesin On");
        } else {
            System.out.println("Mesin Off");
        }
        System.out.println("Kecepatan: " + this.kecepatan);
        System.out.println("=====");
    }

    public String getPlatNomor() {
        return platNomor;
    }

    public void setPlatNomor(String platNomor) {
        this.platNomor = platNomor;
    }

    public boolean getIsMesinOn() {
        return isMesinOn;
    }

    public void setIsMesinOn(boolean isMesinOn) {
        this.isMesinOn = isMesinOn;
    }

    public int getKecepatan() {
        return kecepatan;
    }
}

```

```

    public void setKecepatan(int kecepatan) {
        if (!this.isMesinOn && kecepatan > 0) {
            System.out.println("Kecepatan tidak boleh lebih dari 0 jika mesin off");
        } else if (kecepatan > 100) {
            System.out.println("Kecepatan maksimal adalah 100");
        } else if (kecepatan < 0) {
            System.out.println("Kecepatan tidak boleh bernilai negatif");
        } else {
            this.kecepatan = kecepatan;
        }
    }
}

```

```

public class MotorDemo {
    public static void main(String[] args) {
        Motor motor1 = new Motor();
        motor1.displayStatus();
        motor1.setPlatNomor("B 0838 XZ");
        motor1.setKecepatan(50);
        motor1.displayStatus();

        Motor motor2 = new Motor();
        motor2.setPlatNomor("N 9840 AB");
        motor2.setIsMesinOn(true);
        motor2.setKecepatan(40);
        motor2.displayStatus();

        Motor motor3 = new Motor();
        motor3.setPlatNomor("D 8343 CV");
        motor3.setKecepatan(60);
        motor3.displayStatus();
    }
}

```

3.3 Pertanyaan

1. Pada class MotorDemo, saat kita menambah kecepatan untuk pertama kalinya, mengapa muncul peringatan “Kecepatan tidak bisa bertambah karena Mesin Off!”?

Jawab : Peringatan tersebut muncul karena pada kode program telah ditambahkan logika validasi pengecekan. Validasi ini berfungsi untuk menahan agar atribut kecepatan tidak bisa bernilai lebih dari 0 apabila mesin motor masih dalam kondisi mati (*off*).

2. Mengapa atribut merek, kecepatan, dan statusMesin diset private?

Jawab : Atribut diset menjadi private untuk menerapkan konsep *information hiding* (penyembunyian data). Tujuannya adalah untuk melindungi struktur

internal objek agar atribut tersebut tidak bisa diubah secara acak atau sembarangan dari luar *class*.

3. Apa fungsi dari setter dan getter?

Jawab : Fungsi Setter dan Getter

- **Getter:** Merupakan *method* public yang memiliki *return value*, berfungsi khusus untuk mengembalikan atau membaca nilai dari atribut private.
- **Setter:** Merupakan *method* public tanpa *return value* (void), berfungsi khusus untuk memanipulasi atau mengisi nilai pada atribut private secara terkontrol.

4. Ubah class Motor sehingga kecepatan maksimalnya adalah 100

Jawab :

```
public void setKecepatan(int kecepatan) {  
    .....  
    } else if (kecepatan > 100) {  
        // Jawaban Soal No 4: Batas maksimal 100  
        System.out.println("Kecepatan maksimal adalah 100");  
    }  
    .....  
}
```

5. Ubah class Motor sehingga kecepatan nya tidak boleh nilai negative

Jawab :

```
public void setKecepatan(int kecepatan) {  
    .....  
    } else if (kecepatan < 0) {  
        // Jawaban Soal No 5: Mencegah nilai negatif  
        System.out.println("Kecepatan tidak boleh bernilai negatif");  
    }  
    .....  
}
```

3.4 Percobaan 3 – Constructor

Pada pelajaran sebelumnya, instansiasi objek dari suatu class dilakukan dengan menggunakan syntax `new <NamaClass>()`; misalnya `motor1 = new Motor();` Dengan baris kode tersebut, kita telah menggunakan constructor default yaitu `Motor()` tanpa parameter apapun. Oleh karena itu, setiap nilai atribut pada `motor1` akan bernilai default. Atribut merek yang bertipe string bernilai default null, atribut `isMesinOn` yang bertipe boolean bernilai default false, dan atribut kecepatan yang bertipe integer bernilai default 0. Pada beberapa kasus, kita menginginkan suatu objek dari class tertentu sudah memiliki nilai untuk beberapa (atau seluruh) atribut pada saat objek tersebut dibuat.

Hasil Praktikum

```
public class User {
    public String username;
    public String nama;
    public String email;
    public String alamat;
    public String pekerjaan;

    public User(String username, String nama, String email) {
        this.username = username;
        this.nama = nama;
        this.email = email;
    }

    public void cetakInfo() {
        System.out.println("Username: " + username);
        System.out.println("Nama: " + nama);
        System.out.println("Email: " + email);
        System.out.println("Alamat: " + alamat);
        System.out.println("Pekerjaan: " + pekerjaan);
        System.out.println("=====");
    }
}

public class DemoUser {
    public static void main(String[] args) {
        User user1 = new User("annisa.nadya", "Annisa Nadya",
            "annisa.nadya@gmail.com");
        user1.cetakInfo();
    }
}
```

3.5 Pertanyaan

1. Apa yang dimaksud constructor?

Jawab :

Constructor merupakan sebuah *method* yang digunakan secara khusus untuk menginstansiasi (menciptakan) objek baru dari suatu *class*. Constructor memungkinkan kita untuk langsung memberi nilai awal pada atribut-atribut tertentu tepat saat objek tersebut dibuat.

2. Sebutkan aturan dalam membuat constructor?

Jawab : ada dua aturan wajib dalam mendeklarasikan constructor:

- Nama constructor harus sama persis dengan nama *class*-nya.
- Constructor tidak memiliki *return type* (tipe kembalian) seperti void, String, atau int.

3. Lakukan analisa dan buat kesimpulan apakah constructor bisa bertipe private?

Jawab : Bisa. Sebuah *constructor* bisa diberi *access modifier* private. Analisa: Sesuai dengan aturan enkapsulasi, hak akses private berarti hanya bisa diakses dari dalam *class* itu sendiri. Jika *constructor* diset private, maka *class* lain tidak akan bisa menginstansiasi objek dari *class* tersebut (tidak bisa menggunakan *keyword* new dari luar).

4. Tugas

1. Pada sebuah sistem informasi koperasi simpan pinjam, terdapat class Anggota yang memiliki atribut antara lain nomor KTP, nama, limit peminjaman, dan jumlah pinjaman. Anggota dapat meminjam uang dengan limit peminjaman yang ditentukan. Anggota juga dapat mengangsur pinjaman. Ketika Anggota tersebut mengangsur pinjaman, maka jumlah pinjaman akan berkurang sesuai dengan nominal yang diangsur. Buatlah class Anggota tersebut, berikan atribut, method dan constructor sesuai dengan kebutuhan. Uji dengan TestKoperasi berikut ini untuk memeriksa apakah class Anggota yang anda buat telah sesuai dengan yang diharapkan. Perhatikan bahwa nilai atribut pinjaman tidak dapat diubah secara random dari luar class, tetapi hanya dapat diubah melalui method pinjam() dan angsur()

```
public class TestKoperasi {
    public static void main(String[] args) {
        Anggota anggota1 = new Anggota("111333444", "Donny", 5000000);

        System.out.println("Nama Anggota:" + anggota1.getNama());
        System.out.println("Limit Pinjaman: " + anggota1.getLimitPinjaman());

        System.out.println("\nMeminjam uang 10.000.000 .. ");
        anggota1.pinjam(10000000);
        System.out.println("Jumlah pinjaman saat ini:" + anggota1.getJumlahPinjaman());

        System.out.println("\nMeminjam uang 4.000.000 ... ");
        anggota1.pinjam(4000000);
        System.out.println("Jumlah pinjaman saat ini:" + anggota1.getJumlahPinjaman());

        System.out.println("\nMembayar angsuran 1.000.000");
        anggota1.angsur(1000000);
        System.out.println("Jumlah pinjaman saat ini: " + anggota1.getJumlahPinjaman());

        System.out.println("\nMembayar angsuran 3.000.000");
        anggota1.angsur(3000000);
        System.out.println("Jumlah pinjaman saat ini:" + anggota1.getJumlahPinjaman());
    }
}
```

Hasil Tugas

Jobsheet 3 > J Anggota.java > Anggota

```
1 public class Anggota {
2     private String noKtp;    The value of the field Anggota.noKtp is not used
3     private String nama;
4     private int limitPinjaman;
5     private int jumlahPinjaman;
6
7     public Anggota(String noKtp, String nama, int limitPinjaman) {
8         this.noKtp = noKtp;
9         this.nama = nama;
10        this.limitPinjaman = limitPinjaman;
11        this.jumlahPinjaman = 0;
12    }
13
14    public String getNama() {
15        return nama;
16    }
17
18    public int getLimitPinjaman() {
19        return limitPinjaman;
20    }
21
22    public int getJumlahPinjaman() {
23        return jumlahPinjaman;
24    }
25
26    public void pinjam(int nominal) {
27        if (this.jumlahPinjaman + nominal > this.limitPinjaman) {
28            System.out.println(x: "Maaf, jumlah pinjaman melebihi limit.");
29        } else {
30            this.jumlahPinjaman += nominal;
31        }
32    }
33
34    public void angsur(int nominal) {
35        double minimalAngsuran = this.jumlahPinjaman * 0.10;
36
37        if (nominal < minimalAngsuran) {
38            System.out.println(x: "Maaf, angsuran harus 10% dari jumlah pinjaman.");
39        } else {
40            this.jumlahPinjaman -= nominal;
41        }
42    }
43 }
```

Jobsheet 3 > J Anggotajava > Anggota > angsur(int)

```
1 public class Anggota {
2     private String noKtp;    The value of the field Anggota.noKtp is not used
3     private String nama;
4     private int limitPinjaman;
5     private int jumlahPinjaman;
6
7     public Anggota(String noKtp, String nama, int limitPinjaman) {
8         this.noKtp = noKtp;
9         this.nama = nama;
10        this.limitPinjaman = limitPinjaman;
11        this.jumlahPinjaman = 0;
12    }
13
14    public String getNama() {
15        return nama;
16    }
17
18    public int getLimitPinjaman() {
19        return limitPinjaman;
20    }
21
22    public int getJumlahPinjaman() {
23        return jumlahPinjaman;
24    }
25
26    public void pinjam(int nominal) {
27        if (this.jumlahPinjaman + nominal > this.limitPinjaman) {
28            System.out.println(x: "Maaf, jumlah pinjaman melebihi limit.");
29        } else {
30            this.jumlahPinjaman += nominal;
31        }
32    }
33
34    public void angsur(int nominal) {
35        this.jumlahPinjaman -= nominal;
36    }
37 }
```

Nama Anggota: Donny
Limit Pinjaman: 5000000

Meminjam uang 10.000.000...
Maaf, jumlah pinjaman melebihi limit.
Jumlah pinjaman saat ini: 0

Meminjam uang 4.000.000...
Jumlah pinjaman saat ini: 4000000

Membayar angsuran 1.000.000
Jumlah pinjaman saat ini: 3000000

Membayar angsuran 3.000.000
Jumlah pinjaman saat ini: 0

2. Modifikasi class Anggota agar nominal yang dapat diangsur minimal adalah 10% dari jumlah pinjaman saat ini. Jika mengangsur kurang dari itu, maka muncul peringatan “Maaf, angsuran harus 10% dari jumlah pinjaman”.

```
public void angsur(int nominal) {  
    double minimalAngsur = this.jumlahPinjaman * 0.10;  
  
    if (nominal < minimalAngsur) {  
        System.out.println("Maaf, angsuran harus 10% dari jumlah pinjaman");  
    } else {  
        this.jumlahPinjaman -= nominal;  
    }  
}
```

Link GitHub :

<https://github.com/lovvovala/PrakPBO/tree/main/Jobsheet%203>